Datorlab 1

1 Introduktion till histogram och `scipy.stats`

Syftet med denna lab är att ni ska bekanta er med sannolikhetsfördelningar och histogram. Ett histogram av slumpvärden från en sannolikhetsfördelning ger en uppskattning av dess täthetsfunktion (probability density function).

1.1 Inlämning/svar

Uppgiften ska lämnas in som en python-fil. När frågor ska besvaras med text så ska ni printa frågans nummer i början av svaret; exempelvis sista punkten i deluppgift 2 nedan kan presenteras såhär: `print("2. Sannolikhetsfördelningen verkar vara en normalfördelning")`.

1.2 Sammanfattad Instruktion (detaljerad instruktion nedanför)

Detta är en sammanfattning av allt som ska vara gjort, så att det är lättare att få en översikt och kolla att allt är presenterat. Nedan under "Detaljerad Instruktion" finner ni tydligare anvisningar.

1. Importera biblioteken `scipy.stats`, `matplotlib.pyplot` och `numpy`.
2. Generera en array med slumptal av storlek `N` med hjälp av `scipy.stats.norm.rvs()`.

- Börja med `N = 10` och printa resultatet.
- Från vilken sannolikhetsfördelning tror du vi genererar slumptalen?

1. Låtsas nu att slumptalen är mätresultat från något experiment.

- Sätt `N = 1000`.
- Plotta mätdatan (dvs arrayen du nyss skapat) i en vanlig *xy*-plot och lägg märke till att det mest är "brus".

1. Du ska skapa histogram över mätdatan för att undersöka hur den är fördelad och bekanta dig med histogram. För att skapa ett histogram med `b` binnar använder du `pyplot.hist(array, bins=b)`.

- Skapa histogram över dina mätdata för olika antal binnar `b=10`, `b=30`, `b=100` och `b=N`.
- Vad händer med histogrammet när du ändrar `b` och varför? Lägg märke till hur axlarna förändras.
- Hur många binnar tycker du är "bäst" att använda? Prova att variera antalet mätpunkter `N` också.
- Vad händer med histogrammen om du skapar dem med `density=True`? Var uppmärksam på hur *y*-axeln förändras. *Ledtråd*: Läs dokumentationen för `plt.hist()`!

1. Skapa en tvådimensionell array `A` med slumptal, t.ex. tre rader och `N` kolonner.

- Summera alla rader med `numpy.sum(A,axis=0)`.
- Kontrollera längden på den resulterande vektorn med `len()`.
- Vad betyder `axis`-argumentet i `numpy.sum()`? Vad händer om du använder `axis=1`? Hitta dokumentationen för `numpy.sum()` på egen hand.

Bli inte orolig om ovan känns överväldigande! I den här första labben går vi igenom koden som krävs ganska utförligt samt ger ledtrådar till frågorna. Så om du inte känner att du direkt kan sätta igång med uppgifterna är det bara att fortsätta läsa.

Länkar till relevant dokumentation:

- `scipy.stats`:

https://docs.scipy.org/doc/scipy/reference/stats.html

- `scipy.stats.norm`:

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.norm.html#scipy.stats.norm

- `pyplot.hist()`:

https://matplotlib.org/3.1.1/api/_as_gen/matplotlib.pyplot.hist.html

1.3 Detaljerad Instruktion

1.3.1 scipy.stats

(Uppgift 1.)

I den föregående delkursen (programmering) har ni stött på Python-biblioteken *numpy* och *matplotlib*. I den här delkursen kommer vi utöver dessa att också använda statistikbiblioteket *scipy.stats*.

Observera att *scipy*-biblioteket måste vara installerat för att vi ska kunna använda *scipy.stats*. Men om du redan har använt *numpy* och *matplotlib* har du förhoppningsvis också *scipy*.

På samma sätt som du importerade *numpy* eller *matplotlib.pyplot* i ditt Python-script kan du importera *scipy.stats* på följande vis:

[1]:
```
import scipy.stats as sps
```

`sps` här står för SciPy.Stats, men notera att du kan välja det här namnet själv. Det är bra att välja någonting kort och representativt, för du kommer att skriva det mycket. Du kan också lika gärna skriva

[2]:
```
from scipy import stats as sps
```

De här två sätten att importera biblioteken ska påminna dig om hur du gjorde med *numpy* och *matplotlib.pyplot* i den förra delkursen. Om du inte känner dig bekväm med att importera bibliotek och ge dem ett lokalt namn (lägga dem i ett "namespace") borde du repetera de momenten.

1.3.2 Att generera arrayer med slumptal

(Uppgift 2.)

I den här datorövningen ska du generera arrayer vars element är slumptal. Vi vill att du börjar med att använda följande:

[3]:
```
N = 10
my_random_array = sps.norm.rvs(size=N)
```

Det här ger dig en array med tio element där varje element är ett slumpmässigt reellt tal (du behöver inte ha kallat arrayen `my_random_array`):

[4]:
```
print(my_random_array)
```

```
[-0.46042938  0.26824104  0.47412634  0.5380303  -0.40253281 -1.26633496
 -1.58279552  2.57986943  0.54144669  0.41107062]
```

Kolla på dokumentationen för sps.norm, antingen genom att hitta den på `scipy`s hemsida, eller med någon av dessa metoder:

[5]:
```
# ? sps.norm
# help(sps.norm)
# print(sps.norm.__doc__)
```

Där kan du också få hjälp att besvara följande:

- Från föreläsningarna/boken vet vi nu att slumptal måste vara tagna ur någon bakomliggande sannolikhetsfördelning. Från vilken fördelning hämtar vi våra slumptal?

1.3.3 Histogram

(Uppgift 3.)

Sätt nu `N = 1000`, och plotta igen. Plotta mätdatan (dvs arrayen du nyss skapat) i en vanlig *xy*-plot och lägg märke till att det mest är "brus".

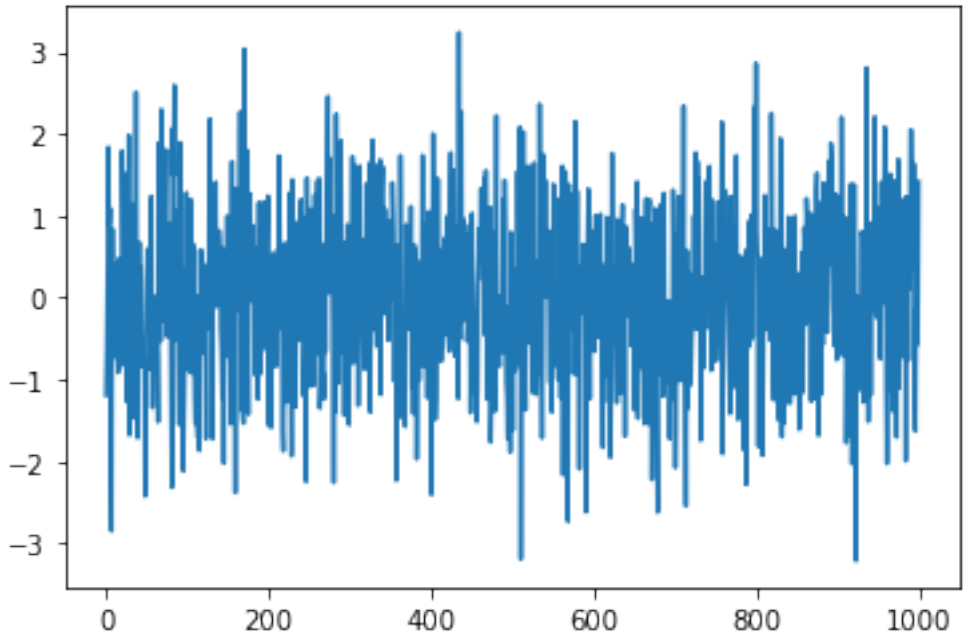
Nu ska vi byta perspektiv och försöka glömma bort att vi har genererat vår array med slumptal från någon sannolikhetsfördelning. Tänk dig istället att arrayen representerar mätdata från något experiment.

Plotta mätdatan i en *xy* -plot genom att importera *pyplot* och anropa:

[6]:
```
from matplotlib import pyplot as plt # Lägg detta i början av ditt script

N = 1000
my_random_array = sps.norm.rvs(size=N)

plt.plot(my_random_array)
plt.show()
```

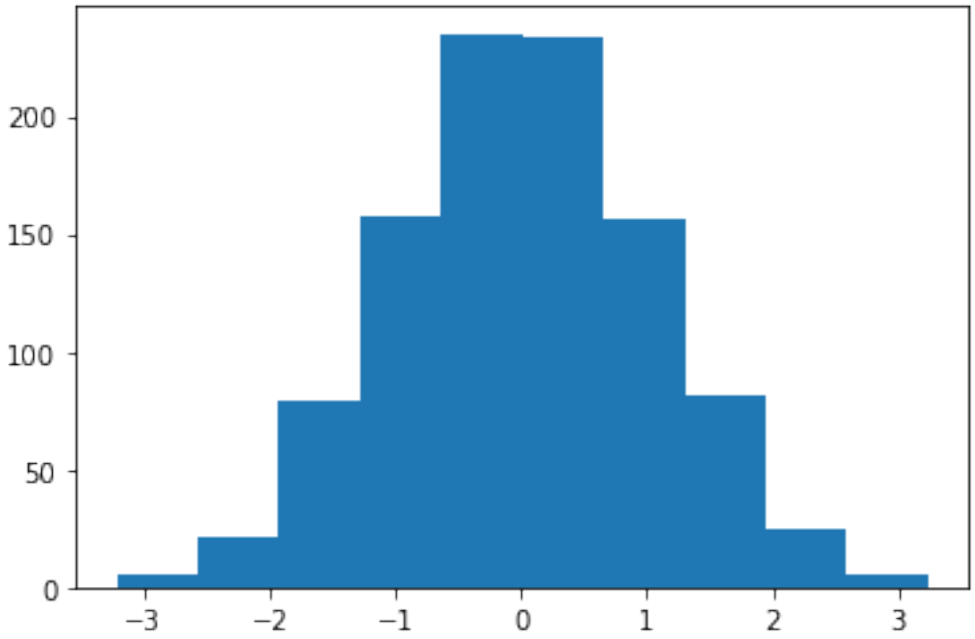


Du bör nu se en “brusig” graf som inte ger så mycket information om mätdatan. Nu vill vi istället använda ett *histogram* för att undersöka hur våra mätdata är fördelade.

Men vad menar vi med fördelad här? Våra mätdata har ju ett största och ett minsta värde, vilket definierar ett intervall i vilket datan ligger. Vi pratar alltså om hur våra mätdata är fördelade på det intervallet.

Skapa ett histogram över de genererade mätdata genom att anropa:

[7]:
```
b = 10 # Antalet binnar i histogrammet
plt.hist(my_random_array, bins=b) # Kom ihåg att du bör ha ändrat storleken
plt.show()                        # på arrayen från N=10 till N>=1000
```



Histogrammet delar upp vårat intervall i ett antal delintervall, eller "bins"/"binnar", i det här fallet tio stycken. Det är intervallet vi ser på x-axeln, och "kanterna" på staplarna ger delintervallen. På y-axeln ger höjden på binnarna *antalet* mätpunkter som hamnat i respektive bin. På det här viset ger histogrammet oss en bild av hur våra mätdata är fördelade.

Nu ska du plotta histogram för olika antal binnar. Skapa histogram för `b = 30`, `b = 100`, och `b = N` och besvara följande för varje `b`:

- Vad händer med histogrammet och varför? Lägg märke till hur axlarna förändras.
- Hur många binnar tycker du är "bäst" att använda? Prova att variera antalet mätpunkter `N` också.

Histogramfunktionen i *pyplot* har ett boolean-argument som heter `density`, se dokumentationen för `plt.hist()`: https://matplotlib.org/3.1.1/api/_as_gen/matplotlib.pyplot.hist.html

Besvara följande:

- Vad händer med histogrammen om du skapar dem med `density=True` ? Testa förslagsvis för `b = 30`. *Ledtråd*: Läs dokumentationen för `plt.hist()`!

1.3.4 Arrayer med fler dimensioner och att summera dessa

Det sista du ska göra är att skapa en array med slumptal som har två dimensioner. I kursen kommer vid flera tillfällen behöva summera slumpvariabler. Det här avsnittet lär dig ett smidigt sätt att skapa flera uppsättningar av slumpvariabler för att sedan lägga ihop dem.

Många funktioner i NumPy/SciPy som skapar arrayer kan ta tupler som storleksargument. Vi skapar en sådan tuple-variabel och använder den för att skapa en tvådimensionell array med slumptal:

[8]:
```
tre_x_N = (3, N) # Tuple för dimensioner
B = sps.norm.rvs(size=tre_x_N)  # skapar en (3 x N)-array B,
                                # dvs B har tre rader och varje rad har N␣
 ↪slumptal.
print(B)
```

```
[[ 0.24163159 -1.37812149 -1.19011617 … -0.07505725  0.32783752
  -0.69775802]
 [ 0.61582966  0.46818221 -0.39810197 …  0.07966613 -2.36106885
  -1.92336365]
 [ 2.49309433 -0.016307   -0.13698231 … -0.59596594  1.04422034
  -0.57153042]]
```

Man kan tänka sig att `B` är som en "behållare" för tre stycken endimensionella arrayer. För att summera dessa tre arrayer på ett snabbt och smidigt vis kan vi använda `np.sum()` i NumPy (kom ihåg att importera NumPy-biblioteket till ditt script!):

[9]:
```
import numpy as np # Igen: ha som god vana att importera i början av ditt script
sumB = np.sum(B, axis=0)
print(len(sumB)) # Skriv ut storleken på resulterande arrayen sumB
```

```
1000
```

Besvara följande:

- Vad betyder `axis`-argumentet i `np.sum()`? Vad händer om du använder `axis=1`? Hitta dokumentationen för `np.sum()` på egen hand.